

Problem statement

2. Autonomous Machine Learning Research Agent for Recommender Systems

In response to some queries, our engineers have provided updates to the problem statement to improve clarity and to support participants better.

Problem Statement last updated: 26 August 2026, 6:33PM

Added downloadable kuairand-starter-kit.zip under '[Starter Kit](#)'

Technical Workshop Webinar with Q&A will be held on **28 Aug, 2:00 to 2:45pm**.

[Click here to join the webinar!](#)

1. Background

Motivation

Machine learning engineers (MLEs) spend much of their time on a single activity: **taking a dataset and a set of metrics, then iterating on a model again and again to push the score higher**. This work is inherently cyclic — every round repeats the same loop, shown in Figure 1.

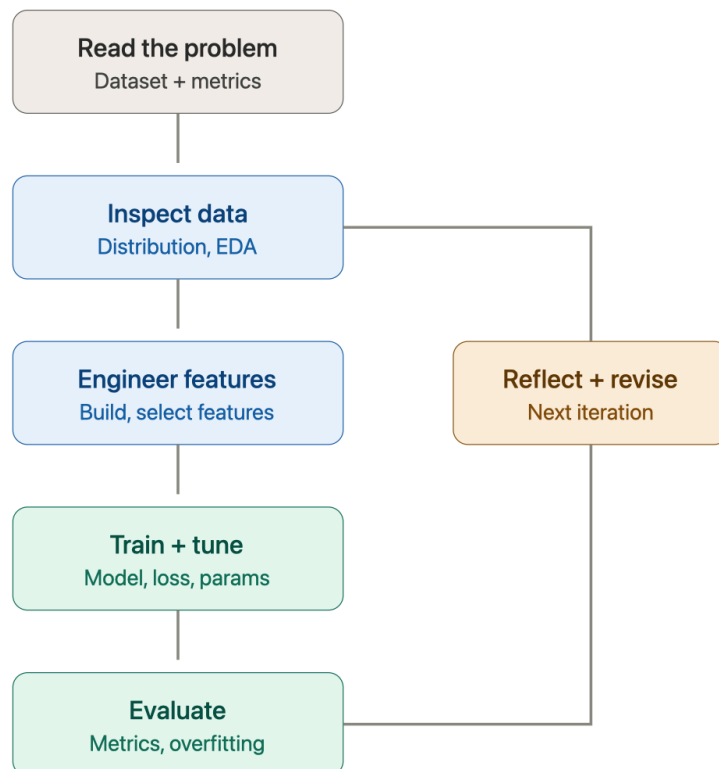


Figure 1. The MLE iteration loop. A closed cycle of five core stages, plus a reflection step that feeds the next round:

1. **Read the problem** — understand the given dataset and the target metrics.
2. **Inspect data** — study data distribution through exploratory data analysis (EDA).
3. **Engineer features** — build and select input features (see Appendix A.5).
4. **Train + tune** — choose a model, set the loss function, and tune hyperparameters.
5. **Evaluate** — read the metrics, check for overfitting, and consult the leaderboard.

The result of the **evaluate** stage drives a **reflect + revise** step, which decides what to change and loops back into the next iteration — re-inspecting the data and adjusting the features. The cycle repeats until the score plateaus.

Two of these stages — **engineer features** and **train + tune** — are carried out almost entirely in code: the engineer writes scripts to transform the data, define the model, and run training. In other words, each turn of the loop produces and modifies code. This is what makes the loop a natural target for automation: it is structured and repeatable, yet writing and revising that code is exactly the kind of task a code-generating LLM can take on.

The loop is also repetitive and mechanical. It draws heavily on "engineering intuition," but many individual steps are well-structured and repeatedly exercised in practice — which is precisely why automating the whole cycle has become an active research direction.

Prior Work

Over the past two years, a new line of work has set out to automate this loop: the **Autonomous ML Research Agent**, an LLM-driven agent that runs the cycle in Figure 1 on its own. It reads the problem, **writes the code** for each stage, trains and evaluates the model, reflects on the results, revises its approach, and finally produces a submission. Representative systems include:

- **MLE-Bench** [1] (OpenAI) — a benchmark of 75 Kaggle competitions, now a standard evaluation suite for such agents.
- **AIDE** [2] (Weco AI) — a state-of-the-art agent that frames ML engineering as code optimization and explores the space of solutions via tree search.
- **AI-Scientist-v2** [3] (Sakana AI) — an end-to-end agent for autonomous scientific and ML research, using agentic tree search to form hypotheses, run experiments, and write up results.

This Challenge

This challenge asks participants to design an **autonomous ML research agent**. Given a public ML dataset and a set of metrics, the agent must **autonomously** run the full loop of Figure 1 — read the problem, engineer features, train and tune the model, evaluate, then reflect and iterate

— to reach the highest possible score across the test sets. Writing the code for each stage is part of the agent's job, not something provided in advance.

New to recommender systems? All benchmarks in this challenge come from the recommendation domain (the KuaiRand family). If terms such as CTR, multi-task learning, NDCG, or Recall@K are unfamiliar, start with the **Appendix: A Primer on Recommender Systems**. At the end of this document — a concept map plus an annotated reading list designed to get you oriented in 1–2 hours.

2. Problem Statement

The Task

Design and implement an Autonomous ML Research Agent. For each benchmark, the agent must autonomously:

1. **Reproduce the official baseline.** Stand up a working end-to-end pipeline and confirm it reaches the official baseline's reported validation score. (The official baseline is a fixed, organizer-provided reference — see *Benchmarks*. Any starter pipeline the agent builds for itself is an internal step, not the reference it is scored against.)
2. **Iterate on the pipeline.** Autonomously draw on established methods from both industry and academia to improve each stage of the pipeline (see Figure 1), and apply those improvements in code. The agent develops using **only the training split and the public validation feedback** — it never has access to the hidden test set.
3. **Improve over the baseline.** Through repeated iterations, drive the **validation** score above the official baseline. Improvement need not be strictly monotonic — as with real-world data, the trajectory may fluctuate — but the agent should show a clear, sustained ability to keep improving relative to the baseline. Final ranking is computed once, on the **hidden test set**, using the submission the agent designates as final.

Task Requirements

1. **Runs end-to-end and aims to beat the baseline.** The agent must run the full pipeline on the required benchmark (KuaiRand-Pure) and reach a converged result; attempting the bonus benchmark (KuaiRand-1k & KuaiRand-27k) is optional. The target is a hidden-test score that exceeds the official baseline; the actual delta achieved — positive or negative — is what feeds into the Primary metric scoring (see Judging Criteria), so falling short of the baseline is scored continuously rather than treated as a disqualifying failure.
2. **Iterates autonomously across the full stack.** The agent should improve the solution on its own, driven by its own evaluation of results. Improvements may target any part of the algorithmic stack — not just the model architecture, but every upstream and downstream module is fair game. The goal is to **minimize human intervention** — a fully autonomous run is the ideal, but a well-instrumented **semi-automated** pipeline that requires only a handful of interventions is an acceptable and realistic outcome; in

practice, we measure how little human intervention a run requires (e.g. the number of manual interventions).

3. **Robust operation.** The pipeline should run reliably with **minimal human intervention**. Robustness here is about how the agent *handles* difficulty, not how often it succeeds — we do not score it by failure count, since a capable agent may fail only on genuinely hard problems. What matters is that when a step fails (a code error, a timeout, an unexpected input), the agent can recover, retry, or route around it, and that long iterative runs neither crash, stall, nor diverge.

3. Constraints & Scope

Category	Constraints & Scope Details
In scope	• Any open-source library or framework (PyTorch, RecBole, TorchRec, LightGBM, ...) • Any papers, public solutions, or pretrained weights • Changes to any pipeline stage — not just the model
Out of scope	• No external training data or pretrained weights trained on these benchmarks' test labels • No hidden-test access during development (train + validation only)
Limits	• KuaiRand-Pure: NDCG@10 / Recall@50, click = positive (fixed) (<i>Required</i>); KuaiRand-1k & KuaiRand-27k: same task and metrics (<i>Bonus</i>) • Hidden test scored once, on the final submission • Compute budget: <i>TBD</i>
Allowed assumptions	• Fixed train / validation / hidden-test split per dataset • Official baseline, scores & evaluation script (incl. convergence rule) • Example submission + output schema

4.

Available Resources & Data

Starter Kit

This content is only supported in a Feishu Docs

To lower the barrier to entry — especially for participants new to recommender systems — the challenge provides a standard starting point. **Download: [kuairand-starter-kit.zip](#)** (above) — numpy only (no torch / pandas / scikit-learn); `python3 baseline.py --model fm` reproduces the official baseline in about 40 s on a single CPU core. It contains:

1. **Fixed data splits:** date-based, taken from the two standard logs (`log_standard_4_08_to_4_21_pure.csv` & `log_standard_4_22_to_5_08_pure.csv`). **train** = date 20220408–20220421 (1,141,112 rows) / **validation** = date 20220422–20220428 (124,909 rows) / **test** = date 20220429–20220508 (170,588 rows). Teams develop on train + validation only; the hidden test set is scored once. Splitting by date rather than by row count avoids any tie-breaking ambiguity on equal timestamps.
2. **Official baseline:** a fixed, organizer-provided reference pipeline shipped in the Starter Kit — a Factorization Machine ($k=16$, $lr=0.001$, 5 categorical fields), numpy only, about 40 s on CPU. Published **hidden-test** scores: GAUC **0.6610** / nDCG@5 **0.5282** / primary **0.5946** (mean over 5 seeds, std 0.0008). Validation: GAUC 0.6674 / nDCG@5 0.5357 / primary 0.6016. Reference rungs for harness self-check — random scoring: primary 0.4753; item popularity: primary 0.5715. Beating *this* baseline is what counts — not a baseline the team builds itself.
3. **Evaluation script:** the exact scoring code (GAUC / nDCG@5) ships in the Starter Kit as `evaluate.py`. It is model-agnostic — it takes only (`user_ids`, `labels`, `scores`), so any model can be scored with it. **Pinned conventions:** users with zero positives count as nDCG = 0 and are included in the average; GAUC counts only users with $0 < \text{positives} < \text{impressions}$, weighted by positive count; nDCG gain = $2^{\text{rel}} - 1$. **Convergence rule:** $\epsilon = 0.002$, $N = 3$ — a run is converged when the validation primary score has not improved by more than ϵ over the last N consecutive iterations ($\epsilon \approx 2.5\sigma$ of the baseline's 5-seed std of 0.0008). The absolute-delta aggregation is unchanged.
4. **Submission format:** a CSV with the header `row_id, user_id, video_id, score`, one line per evaluation-split row. `row_id` is a 0-based, strictly increasing index into the split as produced by `data.load()`; `user_id` / `video_id` are redundant fields used only to verify alignment; `score` is any real number (only the relative order matters), and NaN / Inf are rejected. The `row_id` is required because (`user_id`, `video_id`) is **not unique** in the evaluation split — 3.06% of test rows are repeated pairs, up to 12 times — so it cannot serve as a key. Generate a runnable example with `python3 submit.py --make` and validate with `--check`, which rejects a wrong header, a row-count mismatch, `row_id` gaps, misalignment against the evaluation split, and non-numeric scores.
5. **Run-log requirements:** each iteration should record its **hypothesis**, the **code diff**, the resulting **metrics**, and any **error / recovery events**. These logs are how judges assess **Autonomy** (scored under Impact & Relevance) and **Robustness** (scored under Technical Execution) — see Judging Criteria.

6. **LLM coding agent:** you can use whatever you like, or use [Trae](#) from ByteDance, which provides "Limited offer: new user 7-day free trial".

Benchmarks

KuaiRand-Pure is required and determines 100% of the primary score. **KuaiRand-1k and KuaiRand-27k are bonus datasets** — attempting them is optional and earns extra credit, but neither is required to complete the primary score.

Resource policy. This is a hackathon, so external resources are open by default: use any open-source library (PyTorch, RecBole, TorchRec, LightGBM, ...), read any papers, docs, or public solutions, and use pretrained model weights freely. The agent is expected to draw on whatever published methods it can find — that is what makes it a *research* agent.

There is **one hard rule: no external training data**. Training must rely only on the KuaiRand datasets listed below — no augmenting, joining, or pre-training on any other dataset, and no pretrained model whose weights were trained on these benchmarks' test labels. This single rule is what keeps the hidden-test ranking fair; everything else is unrestricted.

Dataset	Domain & Description	Metrics	Scale
KuaiRand (Kuaishou) Three released variants: KuaiRand-Pure is required, while KuaiRand-1k and KuaiRand-27k are bonus.	Short-video feed. 12 feedback signals (click / like / follow / comment / forward / long_view / play_time ...) plus a randomized-exposure intervention that supports counterfactual evaluation. Relevance label and K are fixed by the organizers (see Starter Kit / TBD): the default task treats click as the positive relevance label and reports NDCG@10 / Recall@50 . The exact label definition and K values are pinned in the Starter Kit so every team solves the same task.	NDCG@10 / Recall@50	Pure: 1.4M interactions (27K users × 7.6K items). 1k: 11.7M. 27k: 322M.

Links: KuaiRand — <https://kuairand.com>

KuaiRand's randomized-exposure data also enables off-policy / counterfactual evaluation (OPE).

5. Deliverables

1. Written Project Description (via Devpost)

- Provide a clear written description of your project that includes:
 - How your solution addresses the problem statement
 - Development tools used (e.g. VSCode, Colab, Jupyter)

- APIs used (e.g. OpenAI GPT-4o, Google Maps API)
 - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
 - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)
2. **Public Code/GitHub Repository**
- Submit a link to a public Code/GitHub repository containing:
 - Well-structured, commented code covering all components of your solution
 - A README file that includes:
 - Project overview
 - Setup and installation instructions
 - Steps to reproduce your results
 - A brief reflection on your solution's limitations and what you would improve given more time
 - Team member contributions (if applicable, i.e. team participants, non-solo participants)
3. **Run & Iteration Logs**
- Submit the per-iteration log required in the Starter Kit (Run-log requirements), covering:
 - Hypothesis for that iteration — what the agent intended to try and why
 - The code diff applied
 - The resulting metrics (NDCG@10 / Recall@50 for the KuaiRand benchmarks)
 - Any error or recovery events encountered, and how the agent handled them
 - A short summary reporting the number of manual interventions during the run (used to assess autonomy per Task Requirement 2)
4. **Final Submission & Results Summary**
- Submit your final model output/checkpoint for the required benchmark (KuaiRand-Pure), in the schema defined by the Starter Kit. If you also attempt the bonus benchmarks (KuaiRand-1k & KuaiRand-27k), submit their outputs as well for bonus scoring.
 - A results table reporting your validation-best score for the required benchmark's metrics (KuaiRand-Pure NDCG@10 / Recall@50), and its absolute delta over the official baseline (per the Evaluation section scoring formula); if you attempted the bonus benchmarks (KuaiRand-1k & KuaiRand-27k), include their NDCG@10 / Recall@50 results as well
 - Reported resource usage required to reach the converged result: total token consumption (input + output) from the agent's LLM calls, and total GPU time (GPU-hours) consumed during training and evaluation (used to score Feasibility & Practicality)

6. Judging Criteria

Judging Criteria	Weight
Technical Execution	35%

Innovation & Problem Insight	20%
Impact & Relevance	20%
Feasibility & Practicality	15%
Presentation & Communication Final Event Only	10%

Technical Execution — Primary Metric & Robustness

Primary metric. We score the **converged result**, not the peak and not the intermediate trajectory. A run is considered converged when **validation score has not improved by more than a small threshold ϵ over the last N consecutive iterations** (default: ϵ and N fixed by the organizers and published in the Starter Kit), or when the run hits the fixed compute/wall-clock budget — whichever comes first. The submission scored for ranking is the **validation-best checkpoint** at that point, evaluated **once on the hidden test set**. The agent develops only on train + validation; it never sees the hidden test set.

- **KuaiRand-Pure is the required benchmark** and determines 100% of the Primary metric score. **KuaiRand-1k and KuaiRand-27k are bonus benchmarks**: a strong result on either earns additional bonus points on top of the Primary metric score, but skipping them does not reduce the KuaiRand-Pure score.
- Per-dataset metrics: **KuaiRand-Pure / KuaiRand-1k / KuaiRand-27k** $\rightarrow$ NDCG@10 / Recall@50. Within each dataset, the score is the **equal-weighted average of each metric's absolute improvement over the official baseline** on the hidden test set. For every metric m :

$$\text{delta}(m) = \text{score_agent}(m) - \text{score_baseline}(m)$$

- $\text{score_dataset} = \text{mean over } m \text{ of } \text{delta}(m)$

Robustness. Not judged by whether the agent ever hits a failure, but by **how it handles one** — recovering, retrying, or routing around a failed step (a code error, a timeout, an unexpected input) so that long iterative runs neither crash, stall, nor diverge before hitting the compute/wall-clock budget.

Innovation & Problem Insight

Judged on what the agent identified as worth trying and why — not on implementation.

- What the agent chose to target across the full algorithmic stack (features, model architecture, training strategy, evaluation loop, etc. — improvements are not limited to the model itself) and the reasoning behind that choice.

- Originality in drawing on published methods, papers, or public solutions — rewarding agents that go beyond naive baseline tweaks.

Impact & Relevance — Autonomy

Autonomy. How much of the improvement loop the agent drives on its own — proposing and testing changes based on its own evaluation of results, not just tuning the model architecture. Measured primarily by the **number of manual interventions** required to reach the converged result; fewer interventions score higher, with fully autonomous runs scoring highest. The fewer humans required, the more this reflects real acceleration of recommender-system R&D.

Feasibility & Practicality — Resource Consumption

How much it costs — in both LLM usage and GPU compute time — to reach the converged result.

- **Token consumption.** Total input + output tokens used by the agent's LLM calls across the run.
- **GPU time.** Total GPU-hours consumed during training and evaluation to reach the converged result — captures the actual compute resources used in a way that wall-clock time alone cannot (e.g. running on more GPUs in parallel looks fast on the clock but is not necessarily cheaper).

7. References

[1] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, L. Weng, and A. Mądry, "MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering," OpenAI, 2024. arXiv:2410.07095.

<https://doi.org/10.48550/arXiv.2410.07095>

[2] Z. Jiang, D. Schmidt, D. Srikanth, D. Xu, I. Kaplan, D. Jacenko, and Y. Wu, "AIDE: AI-Driven Exploration in the Space of Code," 2025. arXiv:2502.13138.

<https://doi.org/10.48550/arXiv.2502.13138>

[3] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha, "The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search," 2025. arXiv:2504.08066. <https://doi.org/10.48550/arXiv.2504.08066>

8. Appendix A. A Primer on Recommender Systems

This appendix gives participants without a recommender-systems background just enough to get started. It is a concept map plus an annotated reading list — not a textbook. Use it to understand the KuaiRand benchmarks and to know what to look up when you get stuck.

A.1 The Big Picture: The Recommendation Pipeline

A modern industrial recommender does not score every item directly. It runs a funnel of stages, each narrowing the candidate set:

Recall → Pre-ranking → Ranking → Re-ranking
millions thousands hundreds final list

- **Recall / Retrieval**: cheaply retrieve a few thousand candidates from millions.
- **Pre-ranking**: a lightweight model trims the candidates further.
- **Ranking**: a heavy, accurate model scores each candidate. **This challenge mostly lives here.**
- **Reranking**: adjust the final ordering for diversity, business rules, and so on.

For this competition you mainly need the **ranking** stage. The KuaiRand benchmarks are ranking/prediction tasks, not full end-to-end pipelines. This content is only supported in a Feishu Docs

A.2 Core Tasks: CTR and the Feedback Funnel

Most industrial ranking is framed as predicting the probability of user feedback:

- **CTR (Click-Through Rate)** — $P(\text{click} \mid \text{impression})$. The user saw the item; will they click?
- **CVR (Conversion Rate)** — $P(\text{conversion} \mid \text{click})$. The user clicked; will they convert (buy)? E-commerce background only; not a task in this challenge.
- **The funnel**: $\text{impression} \rightarrow \text{click} \rightarrow \text{deeper engagement}$ (in e-commerce, $\rightarrow \text{conversion}$). Because these stages are linked, two well-known problems arise:
 - **Sample selection bias**: the post-click signal is only observed on *clicked* items, yet must be predicted for *all* impressions.
 - **Data sparsity**: post-click signals such as `long_view` or `like` are far rarer than clicks.

KuaiRand has no purchase label, so CVR itself is never scored here. But the same two problems reappear on its post-click signals (`long_view`, `like`, `follow` ...), and ESMM-style multi-task modelling — see A.3 — is a legitimate approach to them.

A.3 Multi-Task & Multi-Feedback Learning

Real users produce many signals (click, like, follow, comment, watch-time, and so on). Predicting them jointly — rather than training a separate model per signal — shares representations and tends to improve every task.

- Why it matters here: **KuaiRand** provides **12 feedback signals**, so a multi-task model can learn from several of them jointly even though only `click` is scored.
- The key idea is to balance *shared* parameters (which transfer useful knowledge across tasks) against *task-specific* parameters (which prevent conflicting tasks from hurting one another — the "seesaw" problem).

A.4 Evaluation Metrics

Metric	Intuition	Used for
AUC	Probability that a random positive is ranked above a random negative. Threshold-free and robust to class imbalance.	CTR / CVR prediction in general (not scored in this challenge)
NDCG	Quality of a <i>ranked list</i> , rewarding relevant items near the top (with a position discount).	Ranking quality (KuaiRand)
Recall	Fraction of all relevant items that appear in the returned list.	Coverage (KuaiRand)

Offline vs. online: a higher offline metric does not always mean better real-world performance (because of distribution shift and feedback loops). This competition is evaluated offline, but it is worth knowing the gap exists.

A.5 Feature Engineering Basics

- **ID features:** user ID, item ID, category ID — high-cardinality discrete features.
- **Embedding:** map each discrete ID to a learnable dense vector. This is the foundation of all deep recommenders.
- **Feature crossing:** combine features (e.g. user × category) to capture interactions. Models such as FM and DeepFM automate this.

A.6 Annotated Reading List

[Hints: If you find reading the following material challenging or find you have missing backgrounds, you can use ChatGPT / Claude / ... to explain it to you.]

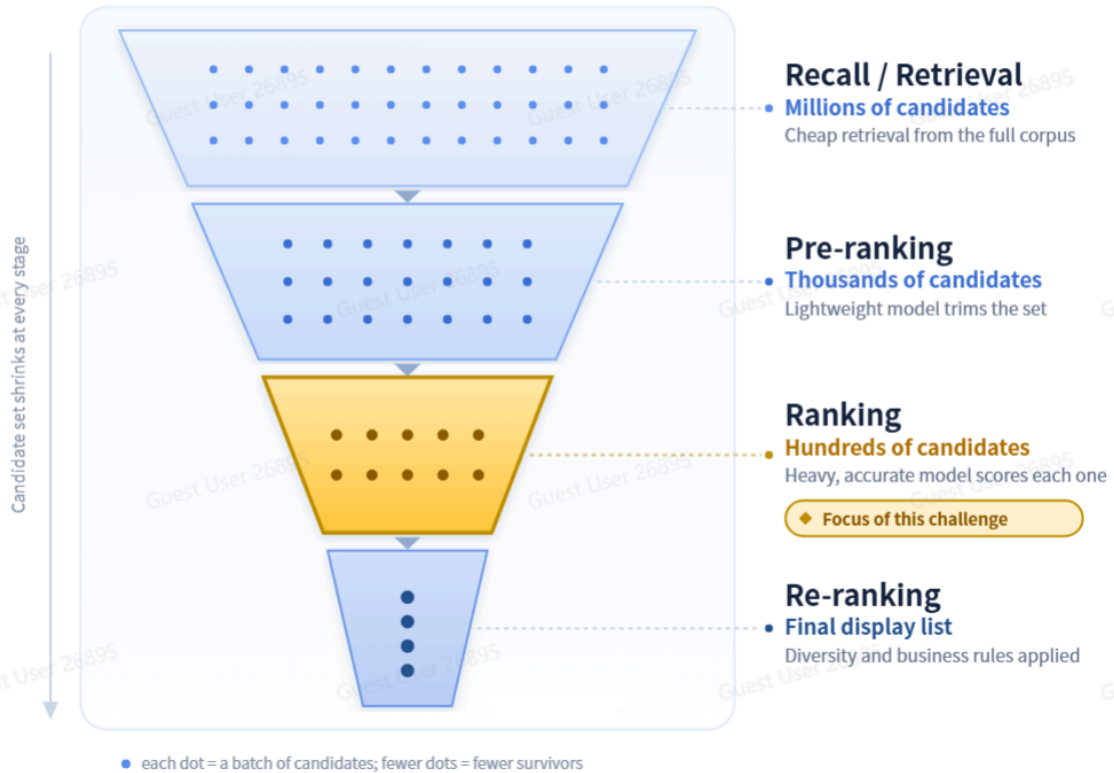
The goal here is only to understand **how a recommender system is structured** — the recall → ranking → re-ranking pipeline — and where the ranking stage (which this challenge targets) sits within it. You do **not** need to read a whole course; the introductory overview is enough.

Read just one of the following:

- Google, *Recommendation Systems* (Machine Learning Crash Course), the **Overview** section —
<https://developers.google.com/machine-learning/recommendation> A short, official overview of the pipeline. Note: Google calls the ranking stage "**scoring**" — this is the same thing as **ranking**, and it is the part this challenge focuses on.
- Wang Shusen, *Recommender Systems*, **Chapter 1 (Overview)** —
<https://github.com/wangshusen/RecommenderSystem> The most beginner-friendly Chinese resource; the first chapter alone gives the full architecture.

The Recommendation Funnel

Millions of candidates in, a short final list out — every stage narrows the set.



Outline

Project Brainstorming: [Project Name — suggestion: "ReflectML" or "AutoRec Agent"]

Team Members: [Fill in] **Submission Deadline:** [Fill in from hackathon rules]

Context

MLEs spend most of their time in one repeating loop: read the problem → inspect data → engineer features → train + tune → evaluate → reflect + revise → repeat. Two stages of that loop (feature engineering, train + tune) are almost pure code, which is why an LLM-driven agent can plausibly run the whole cycle autonomously. This challenge asks us to build exactly that agent, targeted at the KuaiRand-Pure recommendation benchmark (NDCG@10 / Recall@50, click = positive label), with KuaiRand-1k/27k as optional bonus scale-ups.

The organizer-provided Factorization Machine baseline is fixed (hidden-test: GAUC 0.6610 / nDCG@5 0.5282 / primary 0.5946). Our agent has to reproduce that, then autonomously iterate past it — developing only on train + validation, never touching the hidden test set until the one final scored run.

It's worth being explicit about scope here: a production recommender runs a full funnel — Recall/Retrieval (millions of candidates) → Pre-ranking (thousands) → **Ranking** (hundreds, heavy accurate model) → Re-ranking (final list, diversity/business rules). This challenge lives entirely in the **Ranking** stage — we're not building retrieval or re-ranking, just the model that scores each candidate given (user, video) pairs. Keeping the agent's scope pinned to ranking (feature engineering + model + training/tuning against NDCG@10/Recall@50) avoids wasted effort building funnel stages nobody's scoring.

Problem Statement

How might we build an ML research agent that autonomously runs the full read → inspect → engineer → train/tune → evaluate → reflect loop on KuaiRand-Pure, reliably beats the official baseline, and does so with minimal human intervention, bounded compute cost, and full traceability of what it tried and why?

Mission

Build an autonomous ML research agent that:

- Reproduces the official FM baseline end-to-end before touching anything else
- Iterates on every stage of the pipeline (not just model architecture) using published methods, papers, and public solutions
- Detects its own convergence per the organizer's rule ($\epsilon = 0.002$, $N = 3$) and stops cleanly
- Recovers from failures (code errors, timeouts, bad inputs) instead of crashing or stalling
- Logs every iteration's hypothesis, code diff, metrics, and error/recovery events
- Minimizes human intervention, token spend, and GPU-hours to reach its converged result

Target Stakeholders

- **The Agent (autonomous system):** the thing doing the actual work — needs to plan, code, execute, evaluate, and self-correct without a human in the loop.
- **Developer/Operator (us):** needs visibility into what the agent is doing, a safe way to intervene if it's stuck, and confidence it won't burn the compute budget on a bad idea.
- **Judges/Evaluators:** need to see *why* the agent chose what it tried (Innovation & Problem Insight), *how autonomous* the run was (Impact & Relevance), and *what it cost* (Feasibility & Practicality) — not just the final score.

User Stories

Note: the PS was updated 26 Aug 2026 (starter kit link added; technical workshop webinar 28 Aug, 2:00–2:45pm) — worth having someone on the team attend that before locking Phase 1 assumptions.

Agent (Autonomous System)

- As the agent, I want to run `baseline.py` first and confirm I hit the published baseline scores before writing any new code, so I know my harness is trustworthy.
- As the agent, I want to profile the KuaiRand feature space (12 feedback signals, categorical fields) during EDA, so my feature-engineering hypotheses are grounded in the actual data, not generic recipes.
- As the agent, I want to try multi-task formulations (e.g. ESMM/MMOE-style, using auxiliary signals like like/follow/long_view) alongside the primary click task, so I can exploit KuaiRand's 12 signals instead of training on click alone.
- As the agent, I want each iteration to log a hypothesis, a code diff, and resulting metrics *before* I decide on the next change, so my reasoning is auditable and I don't repeat failed strategies.
- As the agent, when a training run throws an error or times out, I want to catch it, log the failure, and retry or route around it, so a single bad step doesn't kill the whole run.
- As the agent, I want to check my validation primary score against the ϵ/N convergence rule after every iteration, so I stop at the right point instead of overfitting to validation or wasting compute.

- As the agent, I want to validate my submission CSV against `submit.py --check` before finalizing, so I never get disqualified over a formatting or alignment error.
- As the agent, I want to track my own cumulative token usage and GPU-hours per iteration, so I can flag when I'm approaching budget and prioritize higher-expected-value experiments.
- As the agent, I want to store each experiment as a hypothesis → method → result → insight chain (not just a score in a list), so I can later ask "what areas of the solution space haven't I explored yet" instead of only "what's my last score."
- As the agent, I want to diagnose *why* a metric moved — e.g. NDCG up but Recall down suggests a ranking problem, train↑/validation↓ suggests overfitting — so my next hypothesis targets the actual weakness instead of guessing.
- As the agent, I want to generate several candidate experiments at once and score each by (expected gain × confidence × novelty ÷ compute cost), so I spend budget on the highest-value idea first instead of running whatever occurs to me next.
- As the agent, I want to smoke-test a new idea on ~1% of the data for one epoch before committing to a full run, and only escalate to 10%, then 100%, if it looks promising, so I kill weak ideas cheaply instead of burning GPU-hours finding out at full scale.
- As the agent, I want most experiments to go through a structured config (model, features, hyperparameters) rather than an LLM rewriting source code, so only genuinely novel ideas cost real tokens and a bad code-gen pass can't silently corrupt the pipeline.

Developer/Operator

- As the developer, I want a live view of the agent's current stage (EDA / feature eng / training / evaluating / reflecting), so I know what it's doing without reading raw logs.
- As the developer, I want to see the full iteration history (hypothesis → diff → metric delta) in one place, so I can assess autonomy and quality after the run without replaying it.
- As the developer, I want an optional manual-intervention hook, so I can nudge the agent if it's stuck in a loop, but I want every intervention I make automatically counted and logged (it costs us on the Autonomy score).
- As the developer, I want a dry-run / small-sample mode, so I can sanity-check a new agent strategy on a data subset before letting it loose on the full 1.14M-row train split.
- As the developer, I want the bonus benchmarks (KuaiRand-1k/27k) to reuse the same pipeline with just a config change, so attempting them isn't a rewrite.

Judge/Evaluator

- As a judge, I want to see *why* the agent tried a given technique (not just that it did), so I can score Innovation & Problem Insight fairly.
- As a judge, I want a clear count of manual interventions and total resource usage, so I can score Autonomy and Feasibility without digging through raw logs.

Key Features / Requirements

Ranked by tier, the same way we want the *agent itself* to rank experiments: **P0 = foundation** (nothing works or scores without it), **P1 = differentiators** (this is where Innovation, Autonomy, and Feasibility points are actually won), **P2 = stretch** (build only if P0/P1 are solid and time remains). "Maps to" names the judging criterion each feature primarily earns.

P0 — Foundation (must exist before anything else matters)

Category	Feature Name	Description	Maps to	Effort	Owner	Status
Core Loop	Orchestrator / Planner	Drives read → inspect → engineer → train/tune → evaluate → reflect; a fixed short list of predefined experiments is fine at this stage — no LLM reasoning yet.	Technical Execution	L		
Modeling	Model Zoo (base)	Pre-built, not LLM-generated: FM (baseline) + DeepFM to start, so the agent selects/configures instead of writing models from scratch.	Technical Execution	M		
Evaluation	Evaluator Wrapper	Wraps organizer's <code>evaluate.py</code> exactly (zero-positive → nDCG 0, GAUC weighting) — never reimplement it.	Technical Execution (correctness)	S		
Evaluation	Convergence Detector	Implements $\epsilon = 0.002$ / $N = 3$ automatically; flags the validation-best checkpoint. Getting this wrong silently changes what gets scored.	Technical Execution	S		

Engineering	Structured Experiment Interface	A config schema (model, features, hyperparams) the agent fills in for ~80% of experiments; LLM only writes raw code for the genuinely novel ~20%.	Technical Execution + Feasibility	M
Observability	Structured Run Log	Auto-generates the required per-iteration log: hypothesis, code diff, metrics, error/recovery events.	Hard deliverable requirement	S
Robustness	Failure Recovery	Catches code errors, timeouts, bad inputs; retries/rolls back/routes around instead of crashing the run.	Robustness (Technical Execution)	M
Submission	Submission Validator	Wraps <code>submit.py</code> <code>--check</code> before finalizing; rejects bad headers/row-id gaps/misalignment automatically.	Avoids disqualification	S

P1 — Differentiators (this is the "not just ChatGPT tuning a learning rate" story)

Category	Feature Name	Description	Maps to	Effort	Owner	Status
----------	--------------	-------------	---------	--------	-------	--------

Research Memory	Research Map / Experiment Tree	Each experiment is a node in a solution tree (not a flat list): a node = a full pipeline config, edges = a proposed modification (draft fresh / debug a failure / improve a working node — the three-mode split AIDE uses). Node metadata stores hypothesis → method → result → insight.	Innovation & Problem Insight	L	Directly modeled on AIDE's solution tree + AI-Scientist-v2's experiment-manager pattern — see "Grounding in Cited Prior Work" below. This is our answer to the Mission's "using published methods, papers, and public solutions."
-----------------	--------------------------------	--	------------------------------	---	---

Research Memory	Metric-Aware Diagnosis Engine	Reads the metric <i>pattern</i> , not just the number: NDCG↑/Recall↓ → ranking problem; train↑/val↓ → overfitting; both low → feature/model mismatch. Turns "score went down, try something else" into an actual diagnosis.	Innovation & Problem Insight	M
-----------------	-------------------------------	---	------------------------------	---

Planning	Best-First Node Selector	A dedicated selector role (separate from the LLM that writes code, per AI-Scientist-v2's experiment-manager split) greedily picks the most promising <i>node</i> to expand next, scored by (expected gain $\times$ confidence $\times$ novelty $\div$ cost) — can branch off an older strong node, not just the most recent result.	Innovation + Feasibility	M	On MLE-Bench, AIDE's tree search won $\sim 4\times$ more medals than a linear (no-branching) agent — direct published evidence this beats a flat "portfolio, pick top one" design.
Efficiency	Multi-Fidelity Runner	Smoke-tests a new idea on $\sim 1\%$ data / 1 epoch, escalates to 10% then 100% only if promising; kills weak ideas before they burn full GPU time. Optionally runs 2–3 branches concurrently at the smoke-test tier (per AI-Scientist-v2's parallel-worker pattern) instead of testing one idea at a time.	Feasibility & Practicality	M	Biggest single lever on GPU-hours — could be worth more than any model-architecture choice.

Data & Features	Multi-Task Feature Exploitation	Uses KuaiRand's 12 feedback signals (like, follow, long_view, forward...) as auxiliary tasks alongside click.	Innovation & Problem Insight	M	Cited directly in the primer (A.3) as a legitimate differentiator over naive tweaks.
Resource Management	Token/GPU Tracker + Budget-Aware Planner	Logs cumulative token/GPU usage per iteration; planner deprioritizes expensive experiments as budget depletes.	Feasibility & Practicality	S–M	Required deliverable numbers (token + GPU-hour totals) come straight out of this.
Evaluation & Convergence	Noise-Aware Convergence Check	Runs 2–3 seeds (or bootstrap-resamples validation) before accepting a result into the research map. Baseline's 5-seed std is 0.0008 vs $\epsilon = 0.002$ — only $\sim 2.5\sigma$ margin, so a single noisy run can look like real improvement or false convergence.	Technical Execution + Innovation	S–M	Protects the research map itself — a false "insight" learned from noise poisons every downstream decision the agent makes off it.
Data & Features	Debiased Learning via Randomized-Exposure Subset	KuaiRand's randomized-exposure intervention subset is unbiased exposure data — usable for inverse-propensity-weighted training or counterfactual evaluation instead of just more rows.	Innovation & Problem Insight	M	Explicitly invited by the primer but not required — most teams will skim past it entirely.

Data & Features	Completion Rate & Rewatch Features	<code>completion_rate = play_time/duration,</code> <code>rewatch_flag = play_time > duration —</code> TikTok has publicly described both as unusually strong signals for its own product.	Innovation & Problem Insight	S	Cheapest, most defensible "grounded in TikTok's own disclosed methodology" claim on the list.
Data & Features	Fast-Skip Implicit Negative Signal	Near-instant skip (<code>play_time</code> below a small threshold) treated as an explicit negative label/auxiliary task, not just "no click" — matches TikTok's disclosed treatment of skips.	Innovation & Problem Insight	S	
Data & Features	Creator-Level Aggregate Features	Rolling engagement-rate features per <code>author_id</code> across their other videos — creator quality is part of TikTok's real ranking story and <code>author_id</code> is already in KuaiRand.	Innovation & Problem Insight	M	

Research Memory	Per-Segment Metric Diagnosis	Buckets users by activity decile (or items by popularity decile) and checks NDCG/Recall per bucket, not just in aggregate — an aggregate +0.01 can hide "better for power users, worse for the long tail."	Innovation & Problem Insight	S–M	Cheap since labels already exist; extends the Metric-Aware Diagnosis Engine rather than replacing it.
--------------------	------------------------------------	--	---------------------------------------	-----	---

Grounding in Cited Prior Work

The PS's Mission line — "iterates on the pipeline... using established methods from both industry and academia" — points at the three papers cited under Prior Work (MLE-Bench, AIDE, AI-Scientist-v2). Here's exactly what we're taking from each, so it's traceable rather than a vague namecheck:

- **AIDE** frames ML engineering as a *solution tree* search, not a flat iteration list: each node is a full pipeline script; a Solution Generator either drafts fresh, debugs a broken node using its traceback, or improves a working node; an Evaluator scores it; a Solution Selector greedily expands the most promising node next. We adopted this directly — it's why Research Map / Experiment Tree and Best-First Node Selector replaced our earlier flat "portfolio selector" draft. AIDE's own paper reports this tree structure winning ~4× more MLE-Bench medals than a linear (non-branching) agent, which is direct published evidence for the switch, not just aesthetic preference.
- **AI-Scientist-v2** separates the *decision* of what to try next (a dedicated "experiment manager agent") from the LLM that actually *writes* the code for it, and runs best-first tree search with multiple parallel exploration workers rather than one strict serial thread. We already had this split (Orchestrator/Planner vs. Code-Gen), so this confirms rather than changes our design — but it does suggest running 2–3 branches concurrently under our Multi-Fidelity Runner (smoke-test several nodes at 1% in parallel, only escalate the winner) instead of always testing one idea at a time. Worth adding as a config knob if time allows.
- **MLE-Bench** isn't a method to borrow features from — it's the standard *evaluation paradigm* this whole task is modeled on (fixed benchmark, agent iterates only on train/validation, scored once on held-out data). Its main value to us is as the source of AIDE's reported 4× number above, and as a sanity check that "tree search beats linear iteration" isn't just our opinion.

Everything else in the P1 table (multi-task feature exploitation, OPE/debiasing, TikTok-specific engineered features) comes from the Appendix primer and public knowledge of TikTok's disclosed ranking signals, not from these three papers — worth keeping that distinction clear in

the written project description, since the deliverable explicitly asks what methods/papers were drawn on.

P2 — Stretch (build only once P0/P1 are solid)

Category	Feature Name	Description	Maps to	Effort	Owner	Status
Research Memory	Research Critic Gate	A cheap pre-flight check (deterministic rules + one lightweight prompt) that can reject an expensive experiment before it runs, with a stated reason (cost, weak prior evidence, etc.).	Innovation polish + Feasibility	M		Don't burn two expensive LLM calls per experiment on this — deterministic checks first, cheap critic prompt second.
Modeling	Extended Model Zoo	Add DCNv2, Wide & Deep, LightGBM once FM/DeepFM are stable.	Technical Execution	M		
Modeling	Hyperparameter Search	Lightweight autotuning (e.g. Optuna) wired into the config interface.	Technical Execution	S–M		

Observability	Live Dashboard	Visualizes current stage + research-map coverage ("Feature Engineering 80%, Sequential Modelling 10%...") for demoing.	Presentation & Communication (final only)	S-M	Doesn't score during development, but this is what makes the final demo land.
Robustness	Checkpointing	Periodic state snapshots so a crash mid-run doesn't lose earlier progress.	Robustness	S	
Bonus	Config-Driven Scale-Up	Same pipeline runs against KuaiRand-1k /27k via config change only.	Bonus points	S	Don't attempt before the required KuaiRand-Pure path is solid.

Extra Features (Idea Bank)

Not folded into the tiered table above — good ideas, but either lower-priority, more speculative, or better named as a limitation than half-built. Worth a scan if P0/P1 finish early.

- **Temporal Drift Check** — train is dated 4/08–4/21, validation 4/22–4/28; explicitly check whether feature distributions shift across that boundary and flag features that "worked" in train but drift in validation. A dataset-specific overfitting guard, not a generic one.
- **Cheap Ensembling of Top Experiments** — right before finalizing, average/stack the top 2–3 distinct experiments' saved predictions. No retraining required, near-free, often a real NDCG bump — good ROI for Feasibility.
- **Plateau Prediction** — fit a simple decay curve to the validation-score trajectory and estimate when further iterations are unlikely to help, instead of only reacting to the ϵ/N rule after the fact. Lets the agent bail early under budget pressure.

- **Session/Sequential Context Features** — rolling features like previous video's engagement or position within a session. TikTok's feed is disclosedly session-aware; this extends the sequential-modelling direction already flagged as underexplored in the research map. If this gets built as a real sequence model rather than hand-rolled rolling features, it's the one place Hugging Face Transformers (a SASRec-style transformer over watch history) would plausibly fit — but that's speculative P2 scope, not a priority; simple rolling/aggregate features are the cheaper first pass.
- **Cold-Start / Exploration Bonus** — sounds compelling but likely doesn't work here: evaluation is offline against fixed historical labels, so an agent can't get credit for "exploring" against a label set that already happened. Better named explicitly as a limitation/future-work item in the README than half-built.
- **"Approximating TikTok's Value Model" framing** — not a feature, a pitch angle: describe the multi-task weighting as the team's approximation of TikTok's disclosed multi-objective ranking blend, rather than presenting it as generic multi-task learning. Costs nothing to build, but sharpens how Innovation & Problem Insight reads to judges who know the product.

Feature Rationale: What the Cited Papers Actually Say, and Why We're Building From Them

The short version lives in the table above; this is the expanded version for anyone drafting the "how your solution addresses the problem statement" section of the written project description, where you'll want to cite these specifically.

AIDE: AI-Driven Exploration in the Space of Code (Weco AI, arXiv:2502.13138)

What it says. AIDE frames ML engineering as a code-optimization problem and formulates the whole trial-and-error loop as a tree search over the solution space. A "solution" is a full script; the empty root is `s0`. Every discovered solution is stored as a node in a solution tree, with edges representing improvement steps. Three components do the work: a **Solution Generator** that proposes new candidates either by drafting from scratch or by modifying an existing node (fixing a bug, or introducing an improvement); an **Evaluator** that runs a candidate and scores it against the objective; and a **Solution Selector** that greedily picks the most promising node to expand next. The paper's framing is explicit that this strategically *reuses and refines* promising solutions rather than rewriting from scratch each round — trading compute for performance. On MLE-Bench (75 real Kaggle competitions), AIDE reportedly won roughly 4× more medals than "the best linear agent" — an agent that iterates on one script sequentially without branching.

Which features this grounds. Research Map / Experiment Tree (a node is a full pipeline config; edges are the modification type), Best-First Node Selector (greedy expansion of the most promising node, not just "rank a batch and run the top one"), and the Structured

Experiment Interface (config-driven changes so most iterations really are cheap "modify an existing node" steps, not full rewrites).

Why we should implement it. This is the one piece of direct, published, quantified evidence in our reference list that a specific architectural choice (tree-structured, branching search) beats the naive alternative (linear, non-branching iteration) on the exact style of task this challenge is. Our earlier draft — a flat portfolio, scored independently, run one at a time — was structurally closer to the "linear agent" AIDE explicitly outperforms. Fixing that before Phase 4 (when the LLM reasoning gets wired in) is cheap now and expensive to retrofit later, since the research-memory schema and the planner's interface both depend on whether experiments are stored as a list or a tree.

The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search (Sakana AI, arXiv:2504.08066)

What it says. An end-to-end agentic system that autonomously formulates hypotheses, designs and runs experiments, analyzes results, and (in their case) writes up manuscripts. Its key upgrade over v1 is dropping the requirement for a human-authored starting code template, and replacing v1's strictly linear hypothesis-testing routine with a **progressive agentic tree search** managed by a dedicated **experiment manager agent** — a role explicitly separated from the LLM call that writes the code for any given node. It implements best-first tree search with configurable parallel exploration workers: e.g. `num_workers=3`, `steps=21` explores up to 21 nodes total, expanding 3 of them concurrently at each step, rather than one strictly serial thread.

Which features this grounds. The existing Orchestrator/Planner vs. Code-Gen split in our P0 tier (this paper is confirming evidence that separation is a deliberate, published design choice, not just tidiness), the Best-First Node Selector (same best-first idea as AIDE, but this paper is explicit about keeping "decide what's next" and "write the code" as separate roles/calls), and the optional parallel-branch mode noted on the Multi-Fidelity Runner.

Why we should implement it. Two concrete, adoptable pieces beyond what AIDE alone gives us. First: don't let the same LLM call that's mid-way through generating code also be the one deciding the tree's strategic direction — a single noisy generation shouldn't get to both propose an idea and approve its own priority. Keeping those as separate calls/roles is cheap and reduces a specific failure mode. Second: running 2–3 branches concurrently at the cheap smoke-test tier of our Multi-Fidelity Runner (each is only 1% data / 1 epoch, so parallelizing is nearly free) surfaces a stronger candidate to escalate to full training than trying ideas one at a time within the same wall-clock budget — a genuine efficiency gain for Feasibility & Practicality, not architectural decoration.

MLE-Bench: Evaluating Machine Learning Agents on Machine Learning Engineering (OpenAI, arXiv:2410.07095)

What it says. A benchmark of 75 real Kaggle competitions used as the standard evaluation suite for ML-engineering agents. It's not an architecture or method paper — it's the yardstick other agents (including AIDE) report their results against.

Which features this grounds. None directly — it isn't a source of design ideas.

Why it still matters. It's the evidentiary backbone behind the AIDE comparison above (the $\sim 4\times$ medal number is an MLE-Bench result), and it's a useful sanity check on our own harness: the challenge's structure here — fixed baseline, develop only on train+validation, score once on held-out test, log every iteration — is essentially MLE-Bench's evaluation paradigm applied to one recommender-systems task instead of 75 Kaggle competitions. Worth citing in the written project description as evidence the evaluation harness follows established practice, not something improvised for this hackathon.

What's explicitly *not* from these three papers

The TikTok-specific engineered features (completion rate, rewatch, fast-skip, creator aggregates), the OPE/debiased-learning idea, and multi-task feature exploitation all come from the Appendix's recommender-systems primer and public knowledge of TikTok's disclosed ranking signals — a different, equally legitimate source of "published methods, papers, and public solutions," just not these three specific references. Worth keeping that distinction explicit in the Devpost write-up (it directly asks what methods/papers you drew on) — naming two separate, deliberate sources of technique (agent-architecture literature vs. recsys domain literature) reads as more careful than blending them into one vague "we used AI research" claim.

Tips — How We Win

- **Most teams lose on plumbing, not ideas.** A large chunk of Technical Execution is just: does the convergence rule match the spec exactly, does the evaluator wrapper match the pinned conventions exactly, does the submission pass `--check` first try. These are boring, easy to get subtly wrong under time pressure, and a wrong implementation silently changes your score without you noticing. The team that wins has an airtight P0 layer, not just clever ideas on top of a shaky one — budget real time to check these against the spec line by line.
- **Depth on 2–3 differentiators beats breadth on ten.** Trying to build the whole idea list is how teams end up with six half-working features instead of two that are genuinely sharp. Highest-ROI pair if forced to choose: **Research Map + Multi-Fidelity Runner** — strongest Innovation story (cheapest to demonstrate convincingly) and strongest Feasibility story (produces the token/GPU numbers you must report anyway).
- **Autonomy is scored by counting, so make the count real, not decorative.** Don't build a "planner" that's actually a rubber stamp on a hardcoded decision tree — if the LLM isn't really making the call, you haven't earned that score. Conversely, don't hide

genuine interventions — a transparent semi-autonomous run beats a run that vaguely claims full autonomy.

- **The demo narrative is worth more than its official 10%.** Presentation is only formally scored at the final event, but it's what separates two teams with similar underlying numbers. Want a clean, replayable story: baseline reproduced → weakness diagnosed → candidates generated and ranked → weak ones rejected with stated reasons → winner smoke-tested cheaply, then scaled up → insight recorded → converged. Consider building the dashboard/log-replay earlier than its P2 slot suggests — it's cheap and it's your closing argument.
- **Don't skip the "boring" deliverables.** README quality, a stated reflection on limitations, team contributions, a correctly-computed results table — these are graded and easy to half-ass at 2am. A technically strong project with a sloppy writeup loses to a modest project with a crisp one, because judges are reading many of these fast.

Technical Architecture

- **Orchestration:** [Agent framework choice — e.g. custom loop, LangGraph, CrewAI]
- **LLM:** [Model choice for planning/code-gen/reflection — kept out of the loop until Phase 4, see roadmap]
- **Execution:** Sandboxed Python execution (numpy-only baseline; pandas for EDA/data handling, PyTorch/RecBole/TorchRec for modeling, scikit-learn for LightGBM's sklearn API + light preprocessing, all allowed once past baseline reproduction)
- **Hugging Face Transformers:** not in scope — no text in this dataset, and the only plausible use (a SASRec-style transformer over session/watch history, see Extra Features) is speculative P2. Not a priority; only pull it in if that specific idea gets built.
- **Data:** Fixed date-based splits per Starter Kit (train 1,141,112 rows / val 124,909 / hidden test 170,588)
- **Evaluation:** Organizer's `evaluate.py`, wrapped, not reimplemented
- **Research Memory:** Structured store (hypothesis → method → result → insight, linked) — the Research Map/Experiment Graph lives here, not just a flat `experiment_history.json`
- **Experiment Interface:** Config schema (model / features / hyperparams / training strategy) the planner fills in; only genuinely novel ideas fall through to raw LLM code generation
- **Multi-Fidelity Runner:** Staged execution (1% → 10% → 100% of train data) with early termination on weak trajectories
- **Logging:** Structured per-iteration log (hypothesis / diff / metrics / errors) → feeds both the deliverable and the dashboard
- **Compute:** [CPU/GPU budget — TBD per organizer limits]

Development Roadmap

Sequenced deliberately so the *dumb but working* version exists before any LLM reasoning is added — a fully autonomous run built on a shaky harness is worse than a semi-automated one built on a solid one.

Phase	Focus Area	Key Tasks
1	Dumb Autonomous Loop	Baseline reproduction, evaluator wrapper, convergence detector; agent picks from a short predefined experiment list (no LLM reasoning yet) — prove <code>python run.py</code> works end-to-end
2	Experiment Infrastructure	Structured experiment folders/config, central experiment log, submission validator wired in — this alone already satisfies most of the Run & Iteration Log deliverable
3	Model Zoo + Tuning	Fill out FM → DeepFM → DCNv2/Wide&Deep/LightGBM, hyperparameter search, multi-task feature exploitation
4	LLM Research Reasoning	Now bring in the planner/strategist: feeds dataset summary + metric history + research map + budget remaining, outputs hypothesis + reasoning + expected effect + cost + priority. Cost-aware portfolio selection and metric-aware diagnosis plug in here.
5	Multi-Fidelity + Early Termination	Staged 1%→10%→100% runner; kill weak experiments early, log GPU saved
6	Recovery, Polish & Bonus	OOM/NaN/timeout/syntax-error handling, research-map dashboard, Research Critic gate if time remains, bonus benchmarks if time remains, README + reflection write-up

Success Metrics

- **Primary metric:** absolute delta over baseline on hidden-test NDCG@10 / Recall@50 (100% of Technical Execution's primary score)
- **Robustness:** run completes without crash/stall/divergence across the full compute budget
- **Autonomy:** number of manual interventions during the run (lower is better; log every one)
- **Cost:** total input+output tokens and GPU-hours to reach the converged result
- **Innovation:** the reflection log itself should show reasoning that goes beyond naive baseline tweaks
- **Demo narrative:** the strongest pitch isn't "we built the smartest AI researcher" — it's a clean, traceable story: baseline reproduced → weakness diagnosed → candidate

experiments generated and ranked → weak ones rejected on cost/gain grounds → promising one tested cheaply first, then scaled up → insight recorded → converged. That sequence alone demonstrates all five judging criteria at once.

Open Questions

- Which orchestration framework are we standardizing on?
- What's our actual GPU budget/access for training runs?
- Do we attempt bonus benchmarks (1k/27k) at all, or spend that time hardening the required KuaiRand-Pure path?
- Who owns the reflection/prompt-engineering for the "next hypothesis" step — that's likely the highest-leverage piece for the Innovation score.

Add ons:

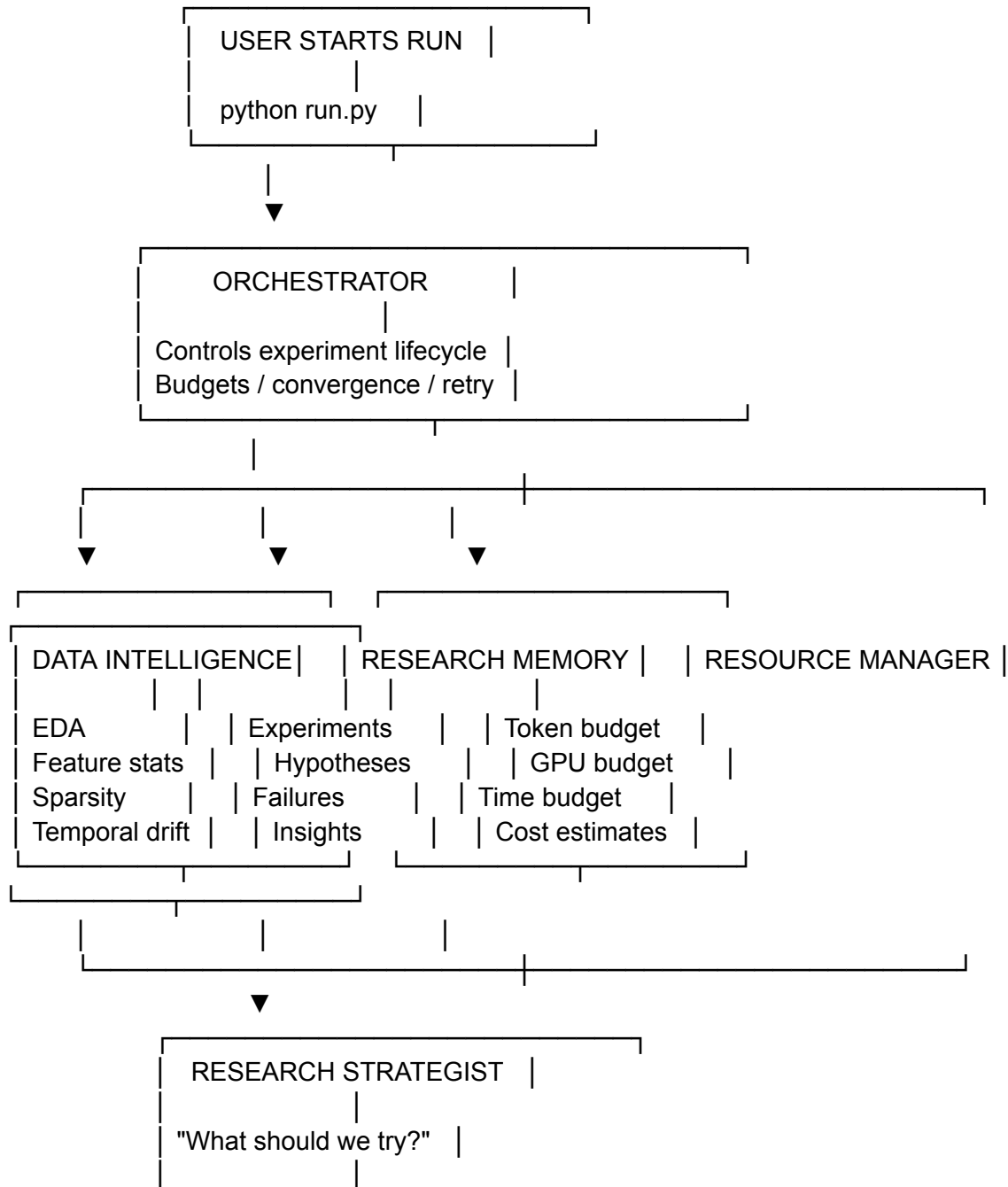
Worth explicitly protecting time for "just try to get a genuinely strong model" rather than assuming sophisticated tooling produces one automatically.

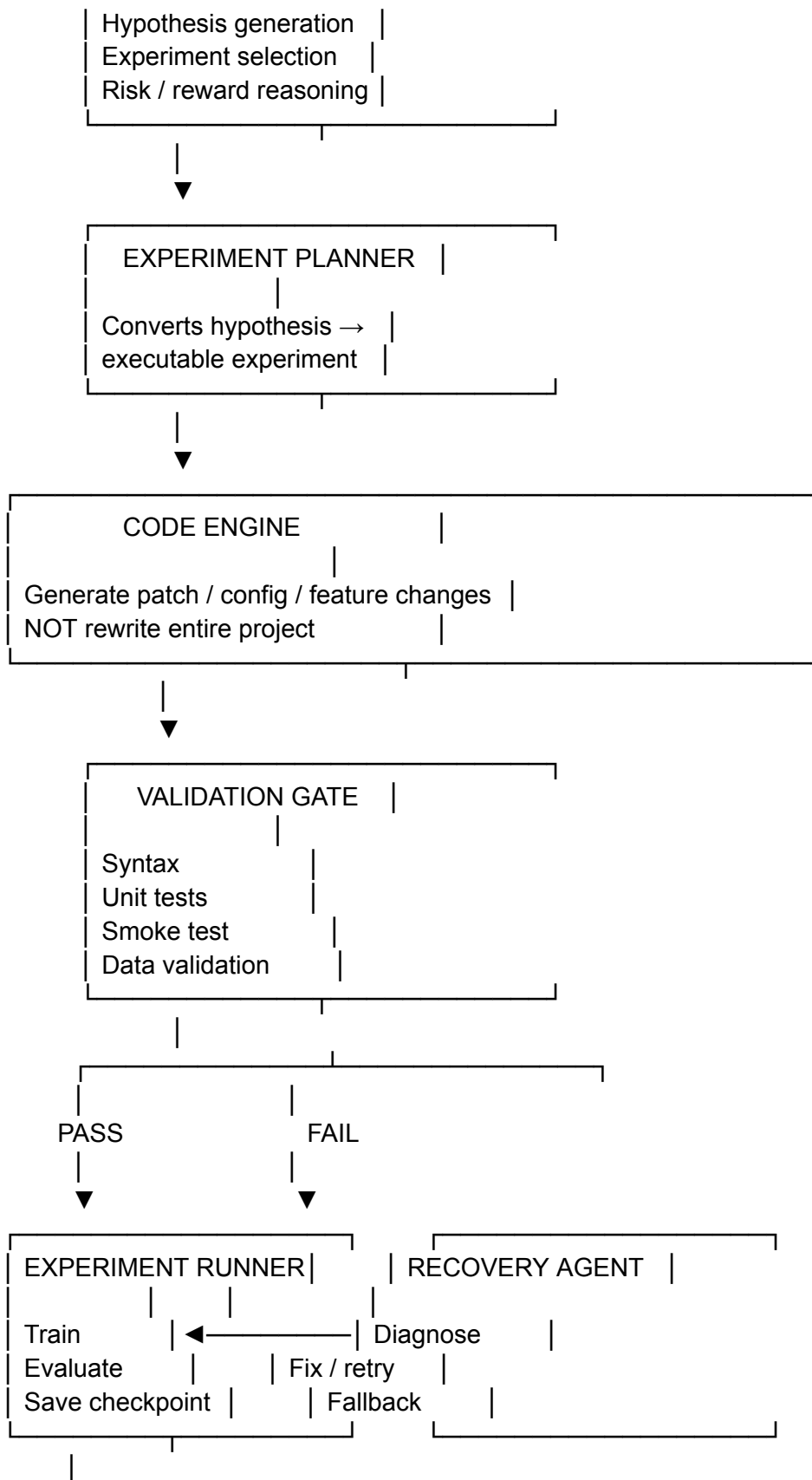
Features to ideate

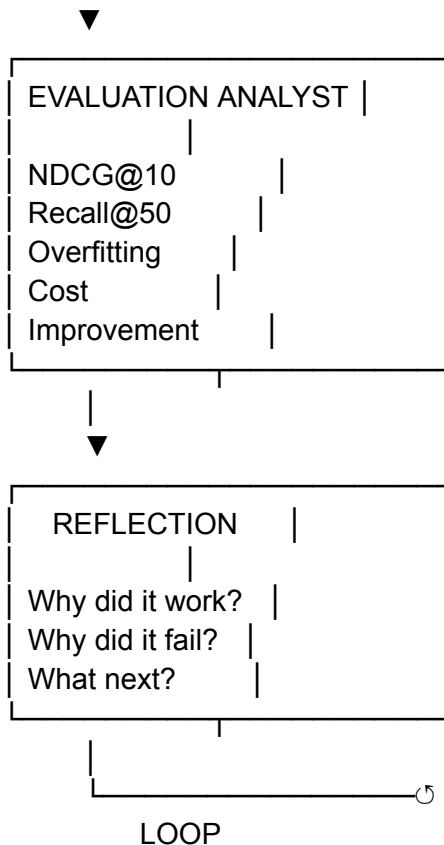
I think the way to make yours genuinely different is to build it as an **experiment-driven research system with a strategic memory and an explicit understanding of why experiments succeeded or failed.**

1. The architecture I would build

Here is the high-level architecture:







That is the **base architecture**.

But this alone won't necessarily make you different. A lot of teams could build something vaguely similar.

The interesting part is **how the Research Strategist decides what to do**.

2. The thing I think can differentiate you: build an Experiment Intelligence System

Most autonomous ML agents will probably do something like:

Experiment 1 → FM

Experiment 2 → DeepFM

Experiment 3 → Tune learning rate

Experiment 4 → Add dropout

Experiment 5 → Try DCN

Experiment 6 → Tune embedding size

Basically:

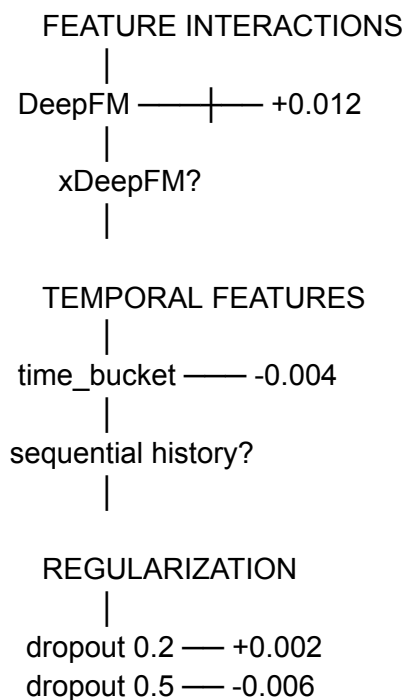
LLM, look at the last score and suggest something.

That is weak.

I would make your agent maintain an actual **research map**.

Instead of just remembering experiments, it learns relationships

For example:



Now the agent isn't just storing:

Experiment 4 scored 0.612.

It stores:

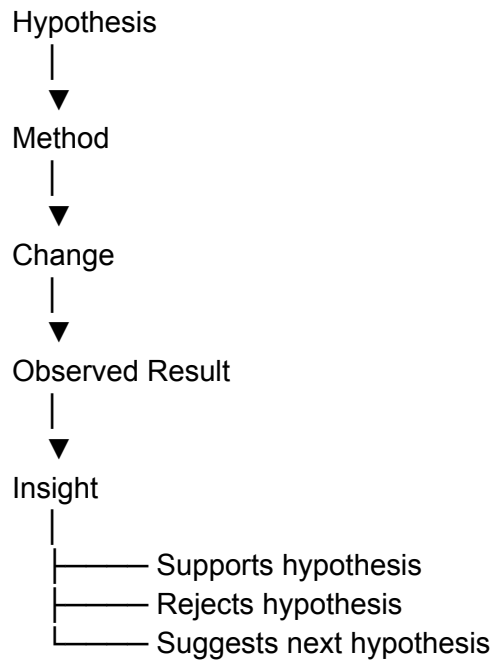
"Moderate regularisation appears beneficial. Excessive regularisation hurts. Temporal bucket features did not help this architecture, but temporal sequence modelling has not yet been tested."

That is much closer to actual research.

3. I would use a Research Knowledge Graph

This could be one of your coolest differentiators.

Each experiment becomes connected to:



For example:

HYPOTHESIS:

FM cannot model sufficiently complex interactions.



METHOD:

DeepFM



RESULT:

NDCG +0.011
Recall +0.008



INSIGHT:
Higher-order feature interactions appear useful.



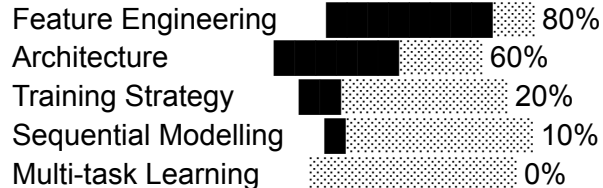
NEXT POSSIBILITIES:

- xDeepFM
- DCNv2
- attention-based feature interactions

This creates a **directed experiment graph** instead of a dumb list of experiments.

Your agent can then ask:

What areas of the solution space have we explored?



That means the agent can deliberately explore **underexplored but promising areas**.

This is much more interesting than random trial-and-error.

4. Give the agent a portfolio of experiments, not one experiment at a time

This is another thing I would do differently.

The agent generates, say:

Candidate experiments:

A. Add time features

Expected gain: +0.002

Cost: Low

Confidence: High

B. Switch DeepFM → DCNv2

Expected gain: +0.010

Cost: Medium

Confidence: Medium

C. Add sequential user history

Expected gain: +0.020

Cost: High

Confidence: Medium

D. Tune embedding dimensions

Expected gain: +0.004

Cost: Low

Confidence: High

Then your system selects using something like:

Experiment Value =

Expected Improvement

× Confidence

× Novelty

÷ Compute Cost

So instead of the LLM going:

 Hmm, maybe try DeepFM.

Your agent explicitly reasons:

DCNv2:

High potential but moderate cost.

Embedding tuning:

Low potential but extremely cheap.

Sequential model:

High potential but expensive.

Decision:

Run embedding tuning first.
If no improvement, allocate budget to DCNv2.

This makes your agent **resource-aware**, which directly addresses the judging criteria.

Remember: they're scoring:

- token usage
- GPU hours
- practicality

So being intelligent about *what not to run* could actually be as important as finding a good model.

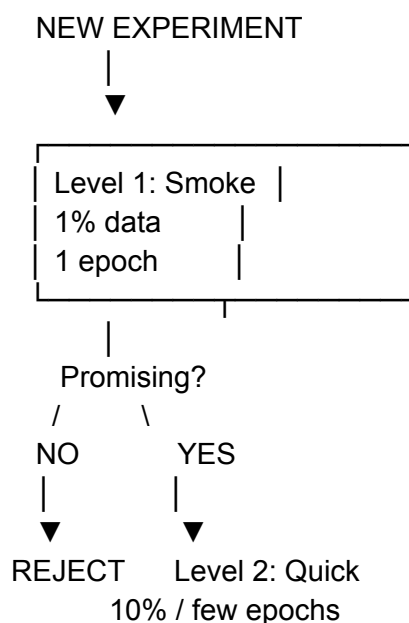
5. Multi-fidelity experimentation: this could be HUGE

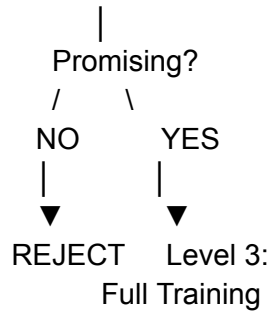
This is one of the biggest things I would add.

Don't run every experiment on the full 1.1M training rows for full training.

That wastes time.

Instead:





This is **how real experimental systems save compute**.

Your agent can kill bad experiments early.

For example:

Experiment: DCNv2

Quick evaluation:

Baseline: 0.6016

Current projected performance: 0.596

Decision:

Terminate experiment early.

GPU saved: 85%.

This could massively improve your **Feasibility & Practicality** story.

6. Build a "metric-aware diagnosis engine"

Don't just give the LLM:

Score = 0.612

Give it an actual diagnosis.

For example:

TRAIN SCORE: 0.82

VALIDATION SCORE: 0.61

→ possible overfitting.

Or:

NDCG@10 ↑
Recall@50 ↓

The agent can interpret:

We are ranking the items we retrieve better, but retrieving fewer relevant items overall.

Then it chooses an appropriate direction.

Metric Pattern → Possible Direction

Low NDCG + Low Recall
→ Representation/model capacity problem

High Recall + Low NDCG
→ Ranking problem

High NDCG + Low Recall
→ Candidate coverage problem

Train ↑↑ / Validation ↓
→ Overfitting

Both low
→ Feature/data/model mismatch

Now your system isn't just:

Score went down, try something else.

It has a **diagnostic policy**.

This can combine deterministic rules with LLM reasoning.

7. I would NOT let the LLM freely rewrite everything

This is where many agent projects become messy.

Instead, create a controlled experiment interface.

For example:

```
Experiment(  
    model="deepfm",  
    embedding_dim=32,  
    hidden_layers=[256, 128],  
    dropout=0.2,  
    features=[  
        "user_id",  
        "video_id",  
        "author_id",  
        "time_bucket"  
    ]  
)
```

Then the agent can modify:

model
features
hyperparameters
training strategy
loss
sampling strategy

through a structured experiment specification.

Only when the agent proposes something genuinely new does it need to modify source code.

This gives you:

80% experiments → cheap structured config
20% experiments → LLM-generated code patches

That means:

- fewer LLM tokens
- fewer broken runs
- better reproducibility
- easier logging

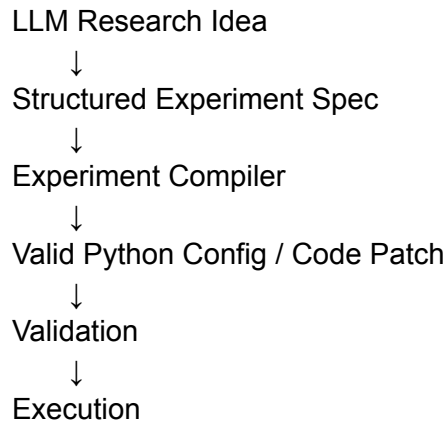
And honestly, **this is a much stronger engineering design** than "let an LLM edit Python files endlessly."

8. Your secret weapon could be an Experiment Compiler

This is something I think could sound very cool in your final presentation.

The LLM doesn't directly create experiments.

Instead:



Example:

```
{
  "hypothesis": "Higher-order feature interactions are under-modelled.",
  "intervention": {
    "type": "architecture",
    "from": "deepfm",
    "to": "dcnv2"
  },
  "budget": {
    "max_epochs": 10,
    "early_stopping": 3
  }
}
```

Your **Experiment Compiler** knows how to translate that into actual code.

That makes your architecture much more reliable.

9. Add a Research Critic

Before running an expensive experiment:

Researcher:

"Let's add a transformer."

↓

Critic:

"Rejected."

Reason:

- Dataset contains limited sequential context.
- Previous temporal features showed weak benefit.
- Estimated compute cost is 5×.
- Expected improvement confidence is low.

This is basically an internal debate, but I wouldn't necessarily use two expensive LLM calls.

You can have:

Research proposal

↓

Deterministic checks

+

Cheap critic prompt

↓

Approve / Reject / Simplify

The key is that every experiment must justify:

Why?

Expected improvement?

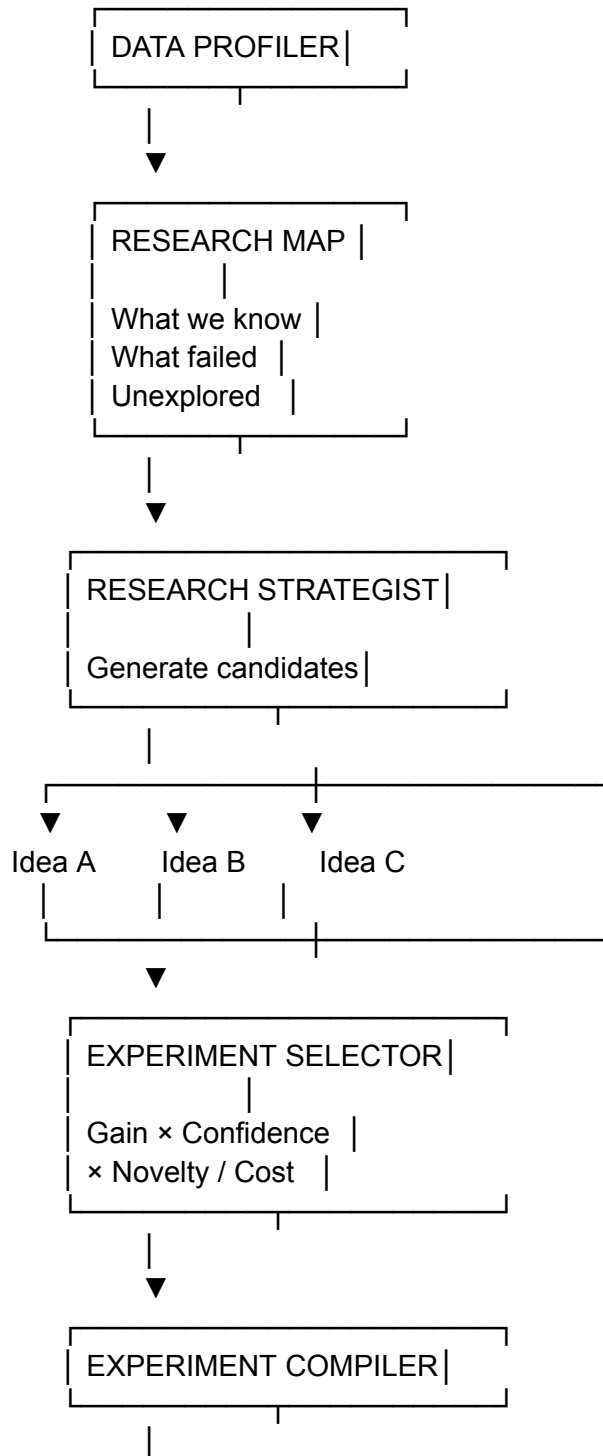
Cost?

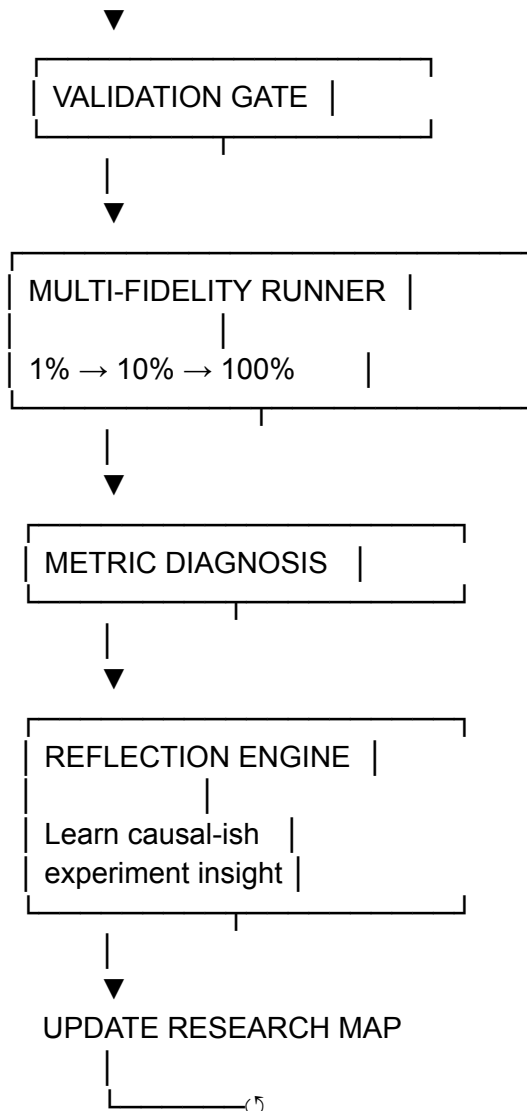
Risk?

What previous evidence supports this?

10. A strong overall system could look like this

AUTONOMOUS RESEARCH AGENT





Phase 1 — Get a dumb autonomous loop working

Goal:

Baseline

- run
- evaluate
- choose from predefined experiments
- run next experiment

No fancy LLM yet.

We prove:

python run_agent.py

works end-to-end.

Phase 2 — Build the experiment infrastructure

Create:

```
experiments/  
  experiment_001/  
    config.json  
    hypothesis.md  
    results.json  
    logs.txt
```

And a central database:

```
experiment_history.json
```

Every experiment automatically records:

- hypothesis
- change
- metrics
- runtime
- GPU usage
- token usage
- errors
- recovery actions

At this point you already satisfy a lot of the deliverable requirements.

Phase 3 — Add baseline model zoo

Don't make the LLM generate every model.

Implement a few strong candidates:

```
FM  
DeepFM  
DCNv2
```

Wide & Deep
LightGBM

Then your agent chooses among them and tunes them.

Later, it can generate new implementations if necessary.

This gives you a stable base.

Phase 4 — Add LLM research reasoning

Only now.

The LLM receives:

Dataset summary
+
Metric history
+
Experiment insights
+
Available models
+
Compute remaining
+
Research map

And outputs:

```
{  
  "hypothesis": "...",  
  "experiment": "...",  
  "reasoning": "...",  
  "expected_metric_effect": {  
    "ndcg": "...",  
    "recall": "..."  
  },  
  "estimated_cost": "...",  
  "priority": 0.87  
}
```

Phase 5 — Add multi-fidelity + early termination

This is when it starts becoming genuinely efficient.

Cheap experiment



Looks bad? → Kill

Looks promising?



More resources



Still promising?



Full training

Phase 6 — Add recovery

Handle:

OOM

NaN

Timeout

Syntax Error

Bad model output

automatically.

What would make your project genuinely stand out?

If we execute this properly, your pitch becomes:

Most autonomous ML agents treat experimentation as a sequence of isolated code-generation tasks. Our agent treats experimentation as a resource-constrained research process.

Then explain:

1. 🧠 Research Map

The agent accumulates **structured scientific memory** rather than raw chat history.

2. 🎯 Metric-Aware Planning

It diagnoses whether the weakness is:

- ranking
- recall
- overfitting
- underfitting

and chooses experiments accordingly.

3. 💰 Cost-Aware Experiment Selection

The agent explicitly optimises:

Expected Improvement

vs

GPU Cost

vs

LLM Token Cost

4. ⚡ Multi-Fidelity Evaluation

Bad experiments are killed early.

5. 🔄 Autonomous Recovery

Failures become part of the research process instead of ending the run.

6. 🧬 Experiment Graph

The agent remembers **relationships between hypotheses and outcomes**, preventing repeated dead ends and identifying unexplored directions.

If I were trying to win with this

My strategy would be:

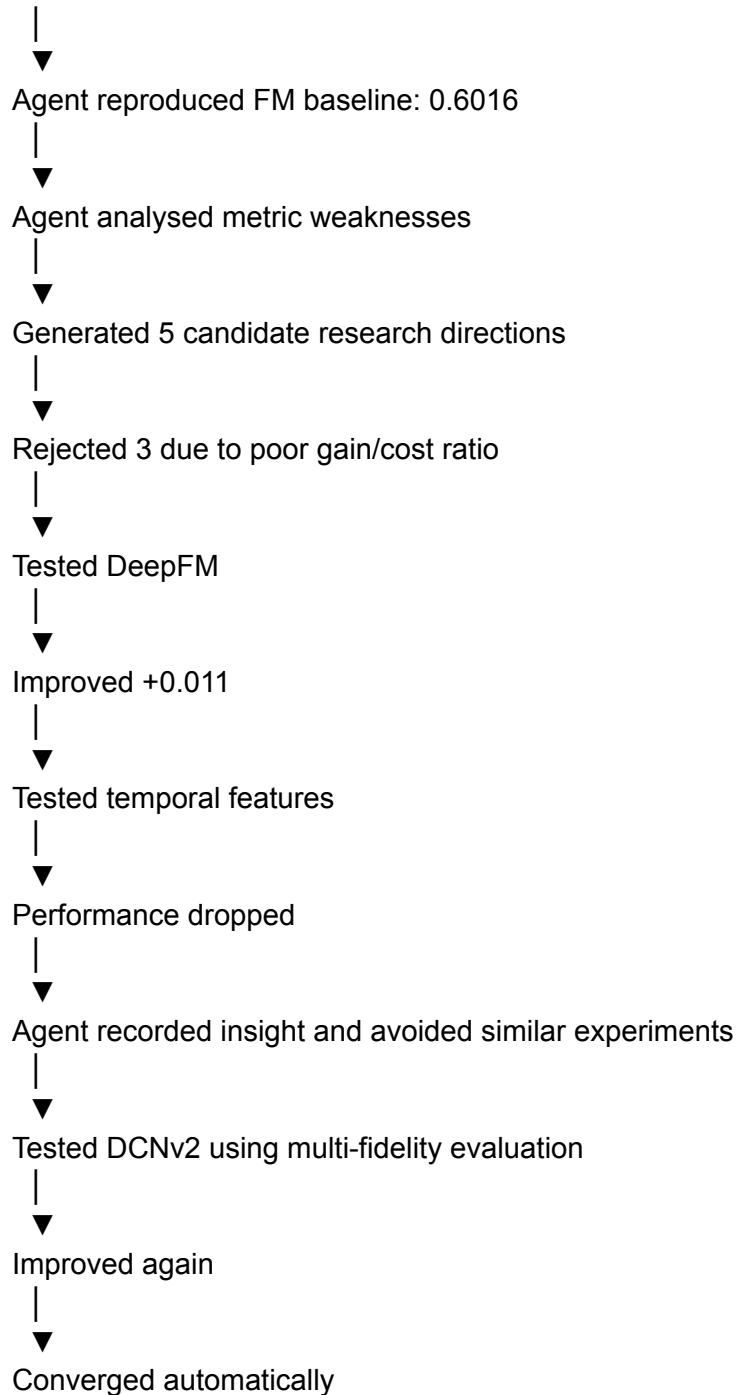
Don't try to claim:

"We built the smartest AI researcher ever."

Instead, build something that is **extremely demonstrable**.

At the final presentation, you should be able to show:

START



That would be **such a strong story** because you're demonstrating all five things the judges care about:

- **Technical execution:** final score
- **Innovation:** research graph + intelligent experiment selection
- **Impact:** minimal human intervention
- **Practicality:** controlled token/GPU usage
- **Presentation:** an extremely clear visual narrative

My recommendation for our next step

We should **not start coding models yet**.

First, let's design the actual project architecture properly into modules and define the **MVP** → **competitive** → **standout** versions.

Then when you get the starter kit, we plug the real dataset/evaluator into that architecture instead of rebuilding everything later.

And yes — I genuinely think there is a way to make this project **more than "ChatGPT runs DeepFM and tunes the learning rate."** The strongest angle is building an agent that learns from its *experiment history* and treats **compute, failure, and exploration itself as things to optimise**. That is where I think your project can become distinctive.

Written Project Description

1. **Written Project Description (via Devpost)**

- Provide a clear written description of your project that includes:
 - How your solution addresses the problem statement
 - Development tools used (e.g. VSCode, Colab, Jupyter)
 - APIs used (e.g. OpenAI GPT-4o, Google Maps API)
 - Libraries and frameworks used (e.g. Hugging Face Transformers, PyTorch, scikit-learn, pandas)
 - Datasets and assets used (e.g. Google Local Reviews dataset, manually labelled data)

Run & Iteration logs

What logs are required — this is a real deliverable (Deliverables section, item 3 in the PS), separate from the checkpoint idea above. For **every iteration** your agent runs, it needs to record:

- **Hypothesis** — what it intended to try and why
- **Code diff** — the actual change applied
- **Resulting metrics** — NDCG@10 / Recall@50 for that iteration
- **Error/recovery events** — anything that broke and how it was handled

Plus a short summary at the end reporting the **total number of manual (human) interventions** during the run — that's the number judges use to score Autonomy.

Links

<https://bytedance.larkoffice.com/wiki/DNtSwxgeciCS2nkiUefc5qqtnkf>

https://github.com/9irija/TikTok_TechJam

<https://kuairand.com/>

<https://tiktoktechjam2026.devpost.com/>